

Cookie based Authentication

What Is Cookie-Based Authentication?

Cookie = small data stored in the browser

When user logs in:

1. Server verifies credentials
2. Server sends back a **Set-Cookie header**
3. Browser stores it
4. Browser automatically sends cookie with every request

What Is Stored Inside Cookie?

Usually:

- A **session ID**
- Or a **JWT**
- Or some auth token

What Is Session-Based Authentication?

This is traditional backend auth.

Flow:

1. User logs in
2. Server creates a session object in memory / Redis / DB
3. Server generates a session ID
4. Session ID stored in cookie
5. On each request:
 - Browser sends cookie
 - Server looks up session in storage

Browser → Login → Server

Server → Creates session → Stores in DB/Memory

Server → Sends sessionId in cookie

Browser → Sends sessionId with every request

Server → Validates sessionId

Cookie vs Session (Clear Difference)

Feature	Cookie Auth	Session Auth
Where is data stored?	Browser	Server
Scalable?	More scalable (JWT)	Needs session store
Stateless?	Yes (if JWT)	No
Can revoke easily?	Hard (JWT)	Easy
More traditional?	No	Yes

Why Use Cookies in Next.js?

Because:

Next.js runs on server AND client

- API routes
- Server components
- Middleware
- SSR

Cookies are automatically sent to the server.

That means:

```
cookies().get("token")
```

Works inside:

- Server components
- Route handlers
- Middleware

No need to manually send Authorization headers.

Why Not Just Use LocalStorage?

Common beginner mistake.

If you store JWT in localStorage:

- ❌ Vulnerable to XSS
- ❌ Not automatically sent with requests
- ❌ Harder to protect

If you use **HttpOnly** cookies:

- ✅ JS cannot access it
- ✅ Auto sent to server
- ✅ Safer

The Real Problem

If you store JWT in localStorage

Then on first page load:

1. Browser requests page
2. Server renders page (SSR)
3. Server CANNOT access localStorage
4. So server doesn't know user is logged in

Because:

- localStorage exists only in browser

- SSR runs on server

So yes —

👉 **SSR does NOT work with localStorage JWT.**

Now the Important Correction

JWT itself is NOT the problem.

Where you store it is the problem.

Case 1: JWT in localStorage

Flow:

Browser → Request page

Server → No token

Server → Renders as guest

Then client hydrates and reads token.

So SSR is broken.

Case 2: JWT in HttpOnly Cookie

Flow:

Browser → Request page

Cookie automatically attached

Server reads cookie

Server verifies JWT

Server renders user page

SSR works perfectly.

Why Header Doesn't Work on First Load?

Because:

When you type:

example.com/dashboard

Browser sends:

GET /dashboard

You cannot manually inject Authorization header there.

Headers are only added when:

- Fetch / Axios requests
- API calls from frontend JS

Not initial navigation.

Why Cookies Fix This

Cookies are automatically attached by browser on:

- First page request
- Every request
- SSR request
- API request
- Middleware

That's why Next.js loves cookies.

Session-based authentication is a **stateful** authentication method where the server stores user session data and sends a session ID to the client (usually via a cookie). On each request, the server looks up the session using that ID.

Cookie-based authentication usually refers to storing a **JWT inside an HttpOnly cookie**. In this case, the server does not store session data. Instead, the token itself contains the user information and is verified on every request, making it **stateless**.

Cookie is not an authentication method.

Cookie is just a transport mechanism.

Both session-based authentication and JWT authentication can use cookies.

The real difference is:

- **Where the authentication state is stored**
 - Session → stored on server
 - JWT → stored inside token

RBAC vs ABAC

RBAC (Role-Based Access Control) grants permissions based on predefined roles assigned to users.

ABAC (Attribute-Based Access Control) grants permissions based on attributes like user properties, resource properties, and environmental conditions.

RBAC is simpler and easier to manage.

ABAC is more flexible and fine-grained.

1 RBAC — Role-Based Access Control

◆ Concept

Access is based on roles.

Example:

- Admin → can delete users
- Editor → can edit posts
- User → can view posts

You assign a role to a user.

Permissions come from the role.

User → Assigned Role → Role Has Permissions → Access Granted

◆ Pros

- Simple to implement
 - Easy to understand
 - Good for most applications
 - Works well in small-medium systems
-

◆ Cons

- Not very flexible
- Role explosion problem

Example:

You need:

- Sales Manager India
- Sales Manager US
- Sales Manager EU
- Junior Sales India

Now roles keep increasing.

ABAC — Attribute-Based Access Control

◆ Concept

Access is based on attributes.

Instead of roles, decisions use:

- User attributes (department, age, clearance level)
- Resource attributes (owner, type, classification)
- Environment attributes (time, location, IP)

If (user.department === "sales")
AND (resource.region === user.region)
AND (time < 6PM)
→ Allow

Access is rule-based, not role-based.

In a document system:

Allow access if:

- User.department = "Finance"
- Document.type = "Budget"
- User.clearance >= Document.sensitivity

No need to create 20 different roles.

◆ Pros

- Extremely flexible
- Fine-grained control
- Scales better in complex orgs

◆ Cons

- Complex to implement
- Harder to debug
- Performance overhead
- Requires policy engine

We prefer HttpOnly cookies over localStorage because cookies prevent JavaScript from accessing the authentication token, which significantly reduces the risk of token theft through XSS attacks.

Additionally, cookies are automatically sent with every HTTP request, which makes them compatible with server-side rendering and simplifies authentication handling on the server.

In contrast, tokens stored in localStorage are accessible via JavaScript and must be manually attached to request headers, increasing security risks and implementation complexity.

CSRF ATTACK

Cross-Site Request Forgery

It is an attack where:

An attacker tricks a logged-in user into performing an unwanted action on a trusted website.

The browser automatically sends cookies.



The server trusts the cookie.



The server cannot tell if the request was intentional.

Because of this browser behavior:

If you are logged into:

bank.com

Your browser automatically attaches your authentication cookie to any request going to bank.com.

Even if that request was triggered from another website.

Real Attack Example

Step 1: User logs into bank

Server sets:

Set-Cookie: token=abc123

Now browser stores it.

Step 2: User visits malicious site

They go to:

evil.com

Hidden inside that page:

```
<form action="https://bank.com/transfer" method="POST">  
  <input type="hidden" name="amount" value="10000">  
  <input type="hidden" name="to" value="attacker">  
</form>
```

```
<script>  
  document.forms[0].submit();  
</script>
```

Step 3: Browser Sends Request

Browser automatically attaches:

Cookie: token=abc123

Bank server sees valid cookie and processes transfer.

💣 Money transferred.

User never intended it.

Important Insight

CSRF does NOT steal your token.

It abuses the fact that:

The browser automatically sends authentication cookies.

That's why CSRF only affects cookie-based authentication.

Why localStorage JWT Doesn't Have CSRF

Because:

Tokens are manually attached via:

Authorization: Bearer <token>

Malicious site cannot access your localStorage.

So it cannot attach the token.

But localStorage has XSS risk instead.

So:

- Cookies → CSRF risk
- localStorage → XSS risk

How We Prevent CSRF

✓ 1 SameSite Cookies

sameSite: "strict"

Prevents cookies from being sent on cross-site requests.

2 CSRF Tokens (Most Secure)

Server generates random token:

csrfToken = randomString

Sends it to frontend.

Frontend must send it back in request header:

X-CSRF-Token: randomString

Attacker cannot know this token.

Server validates it.

3 Double Submit Cookie

Store CSRF token in:

- Cookie
- Request body/header

Server compares both.

A CSRF attack is when a malicious website tricks a logged-in user's browser into sending an unintended request to a trusted website. Since browsers automatically attach authentication cookies, the server may process the request as legitimate. CSRF attacks are typically prevented using SameSite cookies or CSRF tokens.

SameSite cookie attribute

It controls when the browser sends cookies in cross-site requests.

First Understand "Site"

A site = domain (roughly speaking).

Example:

- bank.com
- evil.com

If a request originates from a different domain than the cookie's domain → it's **cross-site**.

Why SameSite Exists

Because of **CSRF attacks**.

Browsers used to send cookies on ALL requests — even from malicious sites.

SameSite gives us control.

1 SameSite = "Strict"

◆ Rule:

Cookie is sent **ONLY if request comes from the same site.**

If request originates from another site → cookie NOT sent.

Example:

User is logged into:

bank.com

Now they click a link from:

evil.com → bank.com

With Strict:

✗ Cookie NOT sent

User appears logged out

Security Level:

Highest

Downside:

Even normal navigation from external sites won't carry cookies.

When To Use:

- Banking
- Highly sensitive apps
- Internal tools

2 SameSite = "None"

◆ Rule:

Cookie is sent on **ALL requests.**

Even cross-site.

Even if triggered from another domain.

Important:

If you use:

sameSite: "none"

You MUST set:

secure: true

Otherwise browser blocks it.

When Used?

- Frontend and backend on different domains
- OAuth login
- Third-party APIs
- Iframes

Example:

frontend.com

api.backend.com

Needs None

3. same site=lax

What Lax Is Trying To Solve

Problem:

- Browsers automatically send cookies.
- Attackers can trigger hidden requests.
- Server trusts cookies.
- CSRF happens.

Strict solved it too aggressively.

None solved nothing.

So Lax was created as a balanced middle ground.

he Core Rule of Lax (Precise Version)

With SameSite=Lax, a cookie is sent if:

1. The request is same-site
OR
2. The request is a cross-site top-level navigation using a safe method (GET)

Everything else → cookie blocked.

That's the exact rule.

Top-level navigation:

👉 When the entire browser page changes to a new URL. COOKIE SENT

Subresource request:

👉 When the current page stays the same but loads something in the background (like fetch, image, script, iframe). COOKIE BLOCKED

When browser prepares a request, it evaluates:

Step 1:

Is the request cross-site?

If NO → send cookie

If YES → go to step 2

Step 2:

Is this a top-level navigation?

If NO → block cookie

If YES → go to step 3

Step 3:

Is the method safe (GET)?

If YES → send cookie

If NO (POST, PUT, DELETE) → block cookie

Why GET Is Allowed

Because HTTP spec says:

GET should be **safe and idempotent**.

Meaning:

- Should not change server state.
- Should only fetch data.

If your GET endpoint deletes or modifies data:

That's a design mistake.

Lax assumes you respect HTTP semantics.

Why Lax is default in modern browsers:

Because most normal user flows are:

- Clicking links
- Navigating pages
- Logging in
- Reading dashboards

All of these are:

Top-level GET requests.

So Lax preserves usability.

if attacker uses:

`<a href="https://mybank.com/deleteAccount">`

And tricks user to click it...

That IS top-level GET.

Cookie WILL be sent.

Which is why:

👉 State-changing operations must NEVER use GET.

Implementing cookie based auth

Step 1 — Install Dependencies

```
npm install express jsonwebtoken bcryptjs cookie-parser cors
```

Step 2 — Basic Server Setup

```
const express = require("express")
const jwt = require("jsonwebtoken")
const bcrypt = require("bcryptjs")
const cookieParser = require("cookie-parser")
const cors = require("cors")
```

```
const app = express()
```

```
app.use(express.json())
app.use(cookieParser())
```

```
app.use(cors({
  origin: "http://localhost:3000", // frontend
  credentials: true
}))
```

```
const SECRET = "supersecret"
```

Step 3 — Fake User (For Testing)

```
const user = {  
  id: 1,  
  email: "admin@test.com",  
  password: bcrypt.hashSync("123456", 10),  
}
```

Step 4 — Login Route (Set Cookie)

```
app.post("/login", async (req, res) => {  
  const { email, password } = req.body  
  
  if (email !== user.email) {  
    return res.status(400).json({ message: "Invalid credentials" })  
  }  
  
  const isMatch = await bcrypt.compare(password, user.password)  
  if (!isMatch) {  
    return res.status(400).json({ message: "Invalid credentials" })  
  }  
  
  const token = jwt.sign(  
    { id: user.id, email: user.email },  
    SECRET,  
    { expiresIn: "1h" }  
  )  
  
  res.cookie("token", token, {  
    httpOnly: true,  
    secure: false, // true in production (HTTPS)  
    sameSite: "lax",  
    maxAge: 60 * 60 * 1000  
  })  
})
```

```
res.json({ message: "Login successful" })
})
```

Step 5 — Auth Middleware (Read From Cookie)

```
const authMiddleware = (req, res, next) => {
  const token = req.cookies.token

  if (!token) {
    return res.status(401).json({ message: "Not authenticated" })
  }

  try {
    const decoded = jwt.verify(token, SECRET)
    req.user = decoded
    next()
  } catch (err) {
    return res.status(401).json({ message: "Invalid token" })
  }
}
```

No Authorization header.

No Bearer token.

Only cookie.

Step 6 — Protected Route

```
app.get("/dashboard", authMiddleware, (req, res) => {
  res.json({
    message: "Welcome to dashboard",
    user: req.user
  })
})
```

Step 7 — Logout Route

```
app.post("/logout", (req, res) => {
  res.clearCookie("token")
})
```

```
res.json({ message: "Logged out successfully" })
})
```

Step 8 — Start Server

```
app.listen(5000, () => {
  console.log("Server running on port 5000")
})
```

If Frontend Exists (Important)

When calling API:

```
fetch("http://localhost:5000/login", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  credentials: "include", // IMPORTANT
  body: JSON.stringify({ email, password })
})
```

Without credentials: "include"
Cookies won't be stored/sent.
